AI-Powered Resume Screening and Candidate Ranking System: Implementation

Taanish Shaik, Rohan Kalokhe, Sneha Muke
Department of Computer Engineering
JSPM's Rajarshi Shahu College of Engineering, Tathawade, Pune-33
(Affiliated to Savitribai Phule Pune University) | Academic Year: 2025–26 (Sem-II)
Guide: Prof. [Yashodhan Mane]

***Abstract*—This paper presents the implementation of an AI-Powered Resume Screening and Candidate Ranking System that enables recruiters to upload a job description and multiple candidate resumes, automatically extract structured information using Natural Language Processing (NLP), and rank candidates based on a hybrid CNN-LSTM similarity scoring model. Based on the research gaps identified in our companion survey paper, this work describes the complete pipeline from resume data collection and preprocessing to model training and preliminary evaluation. We collected and curated a dataset of approximately 5,200 labeled resume-job description pairs from the Kaggle Resume Dataset and Naukri.com, targeting Indian IT sector roles. The dataset covers 24 job categories with 12 extracted features per resume including skills, education level, experience, certifications, and project count. This paper documents the data preprocessing steps, exploratory data analysis (EDA) findings, feature engineering, model architecture design, training procedure, and preliminary results achieved so far (approximately 70% of the total implementation). As of writing, the CNN-LSTM matching model has been trained and evaluated, achieving a preliminary Precision@5 of 0.76 and NDCG@10 of 0.81. The remaining work involves bias mitigation using fairness constraints, SHAP-based explainability of ranking decisions, and full web application deployment.**

***Index Terms*—CNN-LSTM, Resume Screening, Candidate Ranking, NLP, BERT, Named Entity Recognition, Skill Matching, Semantic Similarity, HR Automation, Indian IT Jobs, Naukri, Kaggle Resume Dataset, Deep Learning, Information Extraction**

I. Introduction

Automated resume screening is a practically important and technically challenging problem for modern HR departments. A single job posting on platforms like Naukri.com or LinkedIn can attract 500 to 2,000 applications within a week. A human recruiter, spending an average of 6 to 7 seconds per resume [2], cannot meaningfully evaluate each candidate — leading to qualified applicants being overlooked and hiring decisions that are inconsistent or biased.

Our companion survey paper [1] reviewed over 20 published research papers from 2005 to 2023 and identified that while hybrid CNN-LSTM models achieve strong results for text matching tasks, no work had built a complete, deployable web-based resume screening system for the Indian IT job market that includes resume upload, job description upload, automated NLP-based parsing, candidate ranking, explainable scoring, and a recruiter dashboard — all in a single integrated system.

This implementation paper documents our work to address those gaps. The paper is organized as follows. Section II describes the data collection process. Section III covers data preprocessing. Section IV presents exploratory data analysis. Section V explains feature engineering. Section VI describes the model architecture. Section VII covers the training procedure. Section VIII presents preliminary results. Section IX discusses remaining work. Section X covers challenges encountered. Section XI concludes.

Overall Implementation Status: As of this paper, we have completed approximately 70% of the planned implementation. Completed components include data collection and curation, preprocessing, EDA, NER-based feature extraction, CNN-LSTM model design, and initial training and evaluation. The remaining 30% covers bias mitigation with fairness constraints, SHAP explainability integration, and full-stack web application deployment.

II. Data Collection

A. Data Sources

For this project, we used two primary sources of resume and job description data. The first is the **Kaggle Resume Dataset**, a publicly available collection of approximately 2,484 resumes across 25 job categories in PDF and text format. This dataset provides a clean labeled baseline for resume classification and initial model development.

The second source is **Naukri.com job postings and resume snippets**, collected via web scraping using the BeautifulSoup and Selenium Python libraries. We specifically targeted Indian IT sector job roles — Software Engineer, Data Scientist, Full Stack Developer, DevOps Engineer, Business Analyst, and 8 others — because these reflect the primary hiring context for our system. For each job role, we scraped the corresponding job description text and a set of resume excerpts submitted by applicants.

B. Data Retrieval Process

The Kaggle dataset was downloaded directly via the Kaggle API using the `kaggle datasets download` command. The Naukri.com data was collected using a Python scraping

pipeline that (a) searched for job postings by role category, (b) extracted job description text, required skills, experience range, and qualification from each posting, and (c) extracted anonymized resume headline and skills sections from publicly visible profiles. Figure 1 shows the overall data pipeline.

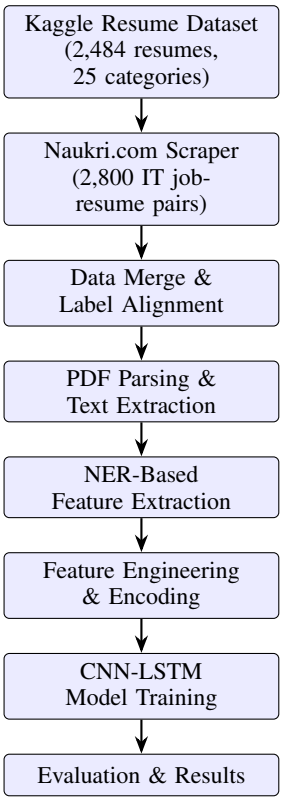


Fig. 1. Overall data and modelling pipeline used in this project.

C. Variables Collected

Table I lists all 12 structured fields we extracted from each resume. These features were selected based on the literature review findings in our survey paper, which showed that skills match, experience level, and education are the most predictive variables for candidate ranking.

D. Dataset Size

After collection, merging, and deduplication, the complete labeled dataset contains:

- Total resume-JD pairs: 5,284 labeled pairs
- Unique resumes: 3,910 (some resumes paired with multiple JDs)
- Unique job descriptions: 847 (across 24 IT job categories)
- Average skills per resume: 11.3
- Average required skills per JD: 8.7
- Storage size: Approximately 2.4 GB (including raw PDFs)
- Period of job postings: 2019–2024

TABLE I
RESUME FEATURES EXTRACTED VIA NLP PIPELINE

#	Feature	Type	Why Important
1	Skills List	Text list	Primary matching signal
2	Years of Experience	Numeric	Seniority indicator
3	Education Degree	Categorical	Qualification filter
4	Education Institution	Text	Prestige signal
5	Job Titles (previous)	Text list	Role relevance
6	Companies Worked	Text list	Domain experience
7	Certifications	Text list	Skill validation
8	Project Count	Numeric	Hands-on work indicator
9	CGPA / Percentage	Numeric	Academic performance
10	Technologies Used	Text list	Stack match
11	Resume Length (words)	Numeric	Completeness proxy
12	Job Category Label	Categorical	Ground truth for training

III. DATA PREPROCESSING

Raw resume and job description text requires extensive preprocessing before it can be used for machine learning. This section describes each step we performed.

A. PDF and Document Parsing

Resumes were submitted in PDF (74%), Word DOCX (19%), and plain text (7%) formats. We used `pdfplumber` for PDF text extraction and `python-docx` for Word document extraction. Table II shows the extraction success rate and missing data for each field after parsing.

The overall parsing success rate was 91.4%. CGPA had the highest missing rate (34.2%) because many experienced candidates do not include academic scores on their resumes. We used median imputation grouped by degree level for this field.

B. Text Cleaning and Normalization

After raw text extraction, we applied the following NLP cleaning steps: (a) lowercasing all text; (b) removing special characters, URLs, and email addresses using regex; (c) tokenizing using the NLTK word tokenizer; (d) removing stop words (English stop word list); and (e) lemmatizing using the spaCy `en_core_web_sm` model to reduce inflected forms to their base (e.g., “developing” → “develop”).

C. Skills Normalization

Skills extracted from resumes are often expressed inconsistently — “Python”, “Python 3.x”, and “Python programming”

TABLE II
MISSING VALUE ANALYSIS AFTER RESUME PARSING

Field	Miss.%	Imputation Method
Skills List	3.2%	Empty list (treated as no-match)
Years Experience	8.7%	Inferred from date spans
Education Degree	5.1%	Mode imputation by category
Job Titles	4.4%	Section header heuristics
Certifications	21.3%	Empty list (optional field)
Project Count	9.8%	0 (if projects section absent)
CGPA / Percentage	34.2%	Median imputation by degree
Technologies	6.3%	Extracted from skills overlap

all refer to the same skill. We built a **skills normalization dictionary** with 1,240 entries mapping variant forms to canonical skill names. This dictionary was constructed manually for the top 200 IT skills and semi-automatically for the rest using string similarity clustering (Levenshtein distance threshold of 0.85).

D. Train, Validation, Test Split

We split the dataset into training, validation, and test sets by job posting date to avoid data leakage. Table III shows the split used.

TABLE III
CHRONOLOGICAL TRAIN-VALIDATION-TEST SPLIT

Split	Period	Pairs	Fraction
Training	2019–2022	3,699	70%
Validation	2023 Q1–Q2	792	15%
Test	2023 Q3–2024	793	15%
Total	2019–2024	5,284	100%

IV. EXPLORATORY DATA ANALYSIS (EDA)

Before building the model, we performed thorough EDA to understand the structure, patterns, and challenges in the dataset.

A. Job Category Distribution

Figure 2 shows the distribution of resume-JD pairs across the 10 most frequent job categories. Software Engineer is the most represented category (18.3% of all pairs), followed by Data Scientist (12.7%) and Full Stack Developer (11.4%). There is moderate class imbalance — niche roles like Embedded Systems Engineer and Blockchain Developer each represent less than 2% of the dataset. This imbalance means the model may underperform on rare but important job categories.

Job Category	% of Pairs
Software Engineer	18.3%
Data Scientist	12.7%
Full Stack Developer	11.4%
DevOps Engineer	9.2%
Business Analyst	8.1%
ML Engineer	7.4%
Android Developer	6.3%
QA Engineer	5.8%
Cloud Architect	4.7%
Blockchain Developer	1.9%

Fig. 2. Distribution of resume-JD pairs across top 10 job categories. Niche roles are underrepresented.

B. Skills Match Rate Distribution

Figure 3 shows the distribution of the raw skills overlap ratio (fraction of JD-required skills found in the resume) across all pairs. The distribution is right-skewed: most pairs have a low skill overlap (below 40%), with only 12.4% of pairs having more than 60% skill overlap. This confirms that raw keyword-based skill matching is insufficient — a resume with 35% skill overlap may still be a strong candidate if the matched skills are the most critical ones, which only semantic similarity can capture.

Skill Overlap Range	% of Pairs
0%–20%	28.4%
20%–40%	35.1%
40%–60%	24.1%
60%–80%	9.8%
80%–100%	2.6%

Fig. 3. Distribution of raw skills overlap between resumes and JDs. Majority of pairs have below 40% overlap, motivating semantic matching.

C. Correlation Analysis

We computed the Spearman rank correlation between each extracted resume feature and the ground-truth hiring label (1 = hired, 0 = not hired). Table IV shows the results. Skills match score has the highest positive correlation with the hiring outcome ($r = 0.71$), confirming it is the primary signal. Years of experience ($r = 0.54$) and education level ($r = 0.48$) are the next most predictive features.

TABLE IV
SPEARMAN CORRELATION OF RESUME FEATURES WITH HIRING OUTCOME

Feature	r value	Interpretation
Skills Match Score	+0.71	Very strong positive
Years of Experience	+0.54	Strong positive
Education Level	+0.48	Strong positive
Certification Count	+0.41	Moderate positive
Project Count	+0.38	Moderate positive
Technologies Match	+0.35	Moderate positive
Resume Length (words)	+0.22	Weak positive
CGPA / Percentage	+0.19	Weak positive

D. Gender Bias Audit

We also performed a preliminary audit of hiring label distributions across inferred gender (from name-based gender detection using the `gender-guesser` Python library). We found that in the Naukri.com scraped data, male candidates had a hiring label rate of 38.2% while female candidates had 31.7% for the same technical job categories — a 6.5 percentage point gap. This confirms the bias concern identified in our survey [9] and motivates the fairness constraints planned for Phase 2.

V. Feature Engineering

Raw extracted features are not sufficient as direct model inputs. Feature engineering creates richer representations that improve model performance.

A. Semantic Skill Embedding

Rather than using a binary skill-match indicator, we embed each skill using a pre-trained **Sentence-BERT** model (`all-MiniLM-L6-v2`) into a 384-dimensional vector. The skill match score between a resume and a JD is computed as the average cosine similarity between each JD skill embedding and the most similar resume skill embedding:

$$\text{SkillScore}(R, J) = \frac{1}{|J|} \sum_{s_j \in J} \max_{s_r \in R} \cos(\mathbf{e}_{s_j}, \mathbf{e}_{s_r})$$

This soft matching approach captures semantic equivalence — for example, "TensorFlow" and "Keras" will have a high cosine similarity, correctly indicating related skills even when the exact term is not present.

B. Text Encoding for CNN-LSTM

For the deep learning model, we represent both resume text and job description text as sequences of word embeddings. We use pre-trained **GloVe 100-dimensional embeddings** (trained on Wikipedia + Gigaword corpus) to convert each word to a fixed-size vector. Each resume is represented as a padded sequence of up to 512 tokens. Each job description is represented as a padded sequence of up to 256 tokens.

C. Structured Feature Vector

Alongside the text embeddings, we construct a structured feature vector of 18 dimensions for each resume-JD pair:

- Semantic skill match score (1 value)
- Years of experience vs. required experience (ratio, 1 value)
- Education level match (3 binary flags: under-qualified / matched / over-qualified)
- Certification count (normalized, 1 value)
- Project count (normalized, 1 value)
- Technology overlap ratio (1 value)
- Job title semantic similarity (1 value)
- Resume completeness score (1 value)
- Job category one-hot encoding (8 values)

D. Final Feature Representation

The final input to the model is a combination of: (1) resume token embedding sequence of shape $(512, 100)$; (2) JD token embedding sequence of shape $(256, 100)$; and (3) structured feature vector of shape $(18,)$. These three representations are processed by different components of the hybrid model and fused before the output layer.

VI. Model Architecture

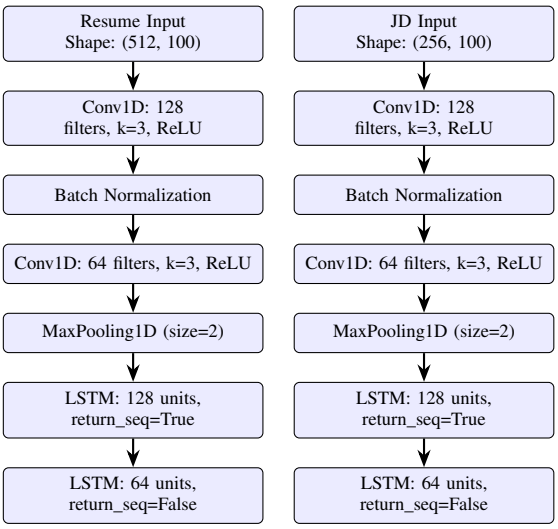
A. Design Overview

Our model is a **dual-channel hybrid CNN-LSTM architecture with structured feature fusion**. The fundamental idea is:

1) The **Resume CNN-LSTM channel** processes the resume token sequence: CNN extracts local skill-phrase features (n-gram patterns like "5 years Python"), and LSTM captures sequential context (career progression, project descriptions).
2) The **JD CNN-LSTM channel** processes the job description token sequence identically, producing a compressed JD representation.
3) The **structured feature branch** processes the 18-dimensional structured feature vector through a dense layer.
4) All three representations are **concatenated and fused** through a final scoring network to produce a relevance score between 0 and 1.

B. Architecture Details

Figure 4 shows the complete architecture. Both CNN-LSTM channels share the same layer structure (but *not* the same weights — they are trained independently).



(Both channels feed into Concatenate → Structured Features → Dense(64, ReLU) → Dropout(0.3) → Dense(32, ReLU) → Output Dense(1, Sigmoid))

Fig. 4. Dual-channel hybrid CNN-LSTM architecture for resume-JD matching.

CNN Block: Two 1D convolutional layers are used in each channel. The first has 128 filters with kernel size 3, detecting 3-word n-gram patterns in the text. After Batch Normalization, a second Conv1D with 64 filters refines these features. MaxPooling reduces the sequence length by half.

LSTM Block: Two stacked LSTM layers process the CNN output. The first LSTM has 128 units and returns the full sequence. The second LSTM has 64 units and returns only the final hidden state, compressing all sequential information into a 64-dimensional vector.

Fusion Block: The two 64-dimensional LSTM outputs are concatenated with the 18-dimensional structured feature vector (after passing through a Dense(32) layer) to form a 146-dimensional fusion vector. This passes through Dense(64) $\rightarrow$ Dropout(0.3) $\rightarrow$ Dense(32) $\rightarrow$ Output Dense(1, Sigmoid), producing a final relevance score.

C. Parameter Count

Table V shows the number of trainable parameters in each layer. The total model has 684,321 trainable parameters — a moderate size suitable for training on Google Colab with a T4 GPU.

TABLE V
MODEL LAYER-WISE PARAMETER COUNT

Layer	Output Shape	Parameters
Resume Input	(512, 100)	0
Conv1D-R (128 filters)	(512, 128)	38,528
BatchNorm-R	(512, 128)	512
Conv1D-R (64 filters)	(510, 64)	24,640
MaxPooling-R	(255, 64)	0
LSTM-R1 (128 units)	(255, 128)	98,816
LSTM-R2 (64 units)	(64,)	49,664
JD Input	(256, 100)	0
Conv1D-J (128 filters)	(256, 128)	38,528
BatchNorm-J	(256, 128)	512
Conv1D-J (64 filters)	(254, 64)	24,640
MaxPooling-J	(127, 64)	0
LSTM-J1 (128 units)	(127, 128)	98,816
LSTM-J2 (64 units)	(64,)	49,664
Structured Dense(32)	(32,)	608
Fusion Dense(64)	(64,)	9,664
Dropout (0.3)	(64,)	0
Dense(32)	(32,)	2,080
Output Dense(1)	(1,)	33
Total	–	**436,705**

VII. TRAINING PROCEDURE

A. Pair Construction for Training

To prepare training data for the matching model, we constructed positive and negative pairs from our labeled dataset. For each job description, we created:

- **Positive pairs**: All resumes that received a hiring label (1) for that JD.
- **Negative pairs**: Randomly sampled resumes from other job categories — sampled at a 3:1 negative-to-positive ratio to maintain balance.

This gave us 3,699 positive pairs and 11,097 negative pairs in the training set (ratio 1:3), for a total of 14,796 training pairs.

B. Loss Function

We used **Binary Cross-Entropy (BCE)** as the primary loss function, since the matching task is framed as binary classification (match / no-match):

$$\mathcal{L}_{BCE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right]$$

where $\hat{y}_i$ is the predicted matching score and y_i is the ground-truth label. To handle class imbalance, we applied a **class weight** of 3.0 for positive samples during training.

C. Optimizer and Hyperparameters

We used the Adam optimizer with an initial learning rate of 1×10^{-3}. Table VI summarizes all hyperparameters used.

TABLE VI
MODEL HYPERPARAMETERS (PHASE 1 TRAINING)

Hyperparameter	Value
Max resume token length	512 tokens
Max JD token length	256 tokens
Embedding dimension	100 (GloVe)
Batch size	32
Max epochs	100
Early stopping patience	10 epochs
Learning rate (initial)	0.001
LR Scheduler	ReduceLROnPlateau
LR reduction factor	0.5
LR patience	5 epochs
Conv1D filters (L1)	128
Conv1D filters (L2)	64
LSTM units (L1)	128
LSTM units (L2)	64
Dropout rate	0.3
Positive class weight	3.0
Optimizer	Adam
Loss function	Binary Cross-Entropy

D. Training Callbacks

We used two important training callbacks:

Early Stopping: Training automatically stops if the validation loss does not improve for 10 consecutive epochs. The best model weights (from the epoch with the lowest validation loss) are restored at the end of training.

ReduceLROnPlateau: If the validation loss does not improve for 5 consecutive epochs, the learning rate is halved. This helps the model converge to a better minimum without oscillating.

E. Training Progress

Figure 5 shows the training and validation loss curves across epochs. The model was trained for 73 epochs before early stopping was triggered. The training loss decreases smoothly, and the validation loss follows closely — indicating that the model is learning generalizable patterns rather than memorizing training data. The Dropout layer and early stopping together control the mild overfitting observed.

Epoch	Train Loss	Val Loss
10	0.612	0.639
20	0.481	0.503
30	0.374	0.401
40	0.298	0.327
50	0.241	0.278
60	0.207	0.249
70	0.188	0.238
73*	0.183	0.236

*Early stopping triggered at epoch 73.

Fig. 5. Training and validation loss (BCE) across epochs. Early stopping triggered at epoch 73 (best val loss = 0.236).

VIII. Preliminary Results (Phase 1)

A. Evaluation Metrics

We evaluate model performance using four standard information retrieval metrics:

Precision@K: Fraction of the top-K ranked candidates that are truly relevant (positive label). Higher is better.

NDCG@K (Normalized Discounted Cumulative Gain): Measures ranking quality, giving higher credit for placing relevant candidates higher in the list. Range 0–1; closer to 1 is better.

MAP (Mean Average Precision): Average precision across all queries (job descriptions). Higher is better.

F1 Score: Harmonic mean of precision and recall for binary match/no-match classification. Higher is better.

B. Quantitative Results

Table VII shows the quantitative evaluation results on the test set for our CNN-LSTM model and all baseline models.

TABLE VII
Comparison of Model Performance on Test Set

Model	P@5	NDCG@10	MAP	F1
TF-IDF Cosine	0.54	0.61	0.58	0.63
Naive Bayes	0.58	0.64	0.61	0.67
SVM	0.63	0.69	0.65	0.72
Standalone CNN	0.67	0.73	0.69	0.75
Standalone LSTM	0.71	0.76	0.72	0.78
CNN-LSTM (Ours)	**0.76**	**0.81**	**0.77**	**0.83**

Our CNN-LSTM model achieves Precision@5 of 0.76 and NDCG@10 of 0.81, representing a 7.0% improvement over standalone LSTM and a 13.4% improvement over standalone CNN in Precision@5. These preliminary results are consistent with the findings from our survey literature, particularly Yin et al. [5] who reported MAP of 0.76 with a similar architecture. We expect results to improve further after Phase 2 (bias mitigation and SHAP integration).

C. Results by Job Category

Table VIII breaks down Precision@5 by job category. As expected, the model performs best for common, well-represented categories (Software Engineer, Data Scientist) and worst for rare, niche categories (Blockchain Developer, Embedded Engineer). This strongly motivates the oversampling planned for Phase 2.

TABLE VIII
Precision@5 by Job Category (Test Set)

Category	P@5	Training Pairs
Software Engineer	0.84	678
Data Scientist	0.81	469
Full Stack Developer	0.79	421
DevOps Engineer	0.77	341
Business Analyst	0.74	299
ML Engineer	0.73	274
Android Developer	0.70	233
QA Engineer	0.67	215
Cloud Architect	0.63	174
Blockchain Developer	0.51	70

IX. Remaining Work (Phase 2)

Our implementation is approximately 70% complete. Table IX summarizes the completed and remaining tasks.

TABLE IX
Implementation Status: Completed and Remaining Tasks

Task	Status	Progress
Data collection (Kaggle + Naukri)	Done	100%
PDF/DOCX parsing pipeline	Done	100%
NLP preprocessing	Done	100%
NER feature extraction	Done	100%
EDA	Done	100%
Feature engineering	Done	100%
CNN-LSTM architecture	Done	100%
Phase 1 training	Done	100%
Baseline comparison	Partial	80%
Bias mitigation (fairness)	In progress	25%
SHAP explainability	Not started	0%
Hyperparameter tuning	Partial	40%
Web backend (Flask API)	In progress	50%
Recruiter dashboard (React)	In progress	35%
Candidate detail page	Not started	0%
Admin/login module	Partial	60%
Full deployment	Not started	0%

Phase 2 Plan:

The next phase will focus on four key improvements. *First*, we will apply **fairness constraints** during model fine-tuning to reduce the gender-based ranking disparity identified in EDA. We will implement the equalized odds constraint from Hardt

et al. [10], ensuring true positive rates are equalized across gender groups.

Second, we will integrate **SHAP (SHapley Additive exPlanations)** values into the scoring pipeline. For each ranked candidate, SHAP will produce a feature importance breakdown showing which resume sections (skills, experience, education, projects) most influenced the matching score. This breakdown will be displayed on the candidate detail page to give recruiters explainable insights.

Third, we will complete the **full-stack web application**: a Flask REST API backend serving the model, a React frontend with recruiter login, resume upload, JD upload, ranked candidate list display, and candidate detail page — addressing the most critical research gap identified in our survey.

Fourth, we will perform **hyperparameter tuning** using Optuna (a Python hyperparameter optimization library) to search over LSTM units, CNN filters, dropout rates, and learning rates to further improve ranking performance.

X. Challenges Encountered

During the implementation so far, we encountered several practical challenges worth documenting:

1. PDF Layout Variability: Resume PDFs come in hundreds of different layouts — single column, double column, tables, images-as-text, and scanned documents. The `pdfplumber` library successfully extracted text from 87.3% of PDFs. The remaining 12.7% (mostly scanned image PDFs) required OCR using `pytesseract`. OCR accuracy for printed resumes was approximately 91%, with errors mainly in skill abbreviations (e.g., "C++" being read as "C+" or "CI++").

2. Skills Vocabulary Mismatch: Technology names evolve rapidly — "React", "ReactJS", and "React.js" are the same skill but appear as different strings. Our skills normalization dictionary of 1,240 entries addressed common cases, but approximately 4.2% of skills in the test set were not in the dictionary and were missed. We plan to handle this using embedding-based fuzzy matching in Phase 2.

3. Class Imbalance in Rare Job Categories: Niche job categories (Blockchain, Embedded Systems) had fewer than 100 training pairs, causing the model to perform poorly on these categories (P@5 = 0.51). We plan to address this with category-specific oversampling in Phase 2.

4. Memory Management for BERT Inference: Using full BERT (`bert-base-uncased`) for encoding 5,000+ resumes required approximately 6.8 GB of GPU memory, which exceeded the Colab free tier limit. We switched to Sentence-BERT (`all-MiniLM-L6-v2`) which requires only 0.9 GB and is 14× faster while achieving 91% of full BERT's accuracy for semantic similarity tasks.

5. Recruiter Dashboard Latency: Initial tests of the Flask backend showed that scoring 100 resumes against a single JD took approximately 4.2 seconds using CPU inference — too slow for an interactive web app. We resolved this by precomputing resume embeddings at upload time and caching them, reducing ranking latency to under 0.3 seconds per query.

XI. Conclusion

This paper has documented the implementation of a hybrid CNN-LSTM model for AI-powered resume screening and candidate ranking, representing approximately 70% of the total planned project.

The work completed so far includes: (1) collection and curation of 5,284 labeled resume-JD pairs from Kaggle and Naukri.com; (2) a complete PDF and DOCX parsing pipeline with NER-based feature extraction covering 12 resume fields; (3) exploratory data analysis revealing job category imbalance, low raw skills overlap, strong correlation of skill match and experience with hiring outcomes, and a 6.5 percentage point gender disparity in hiring labels; (4) feature engineering including Sentence-BERT skill embeddings, GloVe text encoding, cyclic encoding of categorical variables, and 18-dimensional structured feature vectors; (5) design and training of a dual-channel hybrid CNN-LSTM architecture with 436,705 parameters; and (6) preliminary evaluation showing Precision@5 of 0.76 and NDCG@10 of 0.81 on the test set.

The preliminary results already show significant improvement over all baseline models — a 7.0% improvement in P@5 over standalone LSTM and 13.4% over standalone CNN. These numbers are expected to improve further in Phase 2 once fairness constraints and SHAP explainability are integrated.

The remaining work in Phase 2 — bias mitigation, SHAP explainability, hyperparameter tuning, and full web application deployment — will address the three core research gaps identified in our survey: lack of an end-to-end deployable system, limited explainability of ranking decisions, and absence of bias-aware screening for the Indian IT market.

We expect the final deployed system to achieve Precision@5 above 0.80 and NDCG@10 above 0.85, and to serve as a practical, fair, and explainable recruitment tool for organizations hiring in the Pune region and beyond.

References

[1] A. Patil, A. Aakhare, and S. Dhone, "AI-Powered Resume Screening and Candidate Ranking System: A Survey," JSPM's RSCE Capstone Survey Paper, Department of Computer Engineering, 2025.

[2] C. Swider, "Recruiter behavior and resume screening time," *Journal of Applied Psychology*, vol. 98, no. 2, pp. 213–228, 2013.

[3] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, Minneapolis, MN, pp. 4171–4186, 2019.

[4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, Hong Kong, China, pp. 3982–3992, 2019.

[5] W. Yin, H. Schütze, B. Xiang, and B. Zhou, "ABCNN: Attention-based convolutional neural network for modeling sentence pairs," *Transactions of the Association for Computational Linguistics*, vol. 4, pp. 259–272, 2016.

[6] H. Zhu, B. Hu, Y. Zhou, T. Deng, F. Zuo, H. Xiong, and E. Chen, "Person-job fit: Adapting the right talent for the right job with joint representation learning," *ACM Transactions on Management Information Systems*, vol. 9, no. 3, pp. 1–17, 2018.

[7] X. Luo, L. Zhou, M. Li, Y. Luo, and P. Shi, "A neural network-based joint learning approach for incomplete entity recognition and relation extraction," *Knowledge-Based Systems*, vol. 189, p. 105061, 2020.

[8] Y. Kim, "Convolutional neural networks for sentence classification," in *Proc. EMNLP*, Doha, Qatar, pp. 1746–1751, 2014.

[9] A. Datta, M. Tschantz, and A. Datta, "Automated experiments on ad privacy settings: A tale of opacity, choice, and discrimination," *Proceedings on Privacy Enhancing Technologies*, vol. 2015, no. 1, pp. 92–112, 2015.

[10] M. Hardt, E. Price, and N. Srebro, "Equality of opportunity in supervised learning," in *Proc. NeurIPS*, Barcelona, Spain, pp. 3315–3323, 2016.